



UNIVERSIDADE FEDERAL DO AMAZONAS

Instituto de Computação

Disciplina: Redes Sem Fio

**ATIVIDADE — SCAN DE REDES WIFI
ESCANEAMENTO COM ESP32 E ANÁLISE DE RSSI**

Álison de Oliveira Venâncio

Ayrton Finicelli Lemes

Matheus Serrão Uchôa

Victor Ronald

Manaus – AM

2026

**ÁLISON DE OLIVEIRA VENÂNCIO, AYRTON FINICELLI LEMES,
MATHEUS SERRÃO UCHÔA, VICTOR RONALD**

**ATIVIDADE — SCAN DE REDES WIFI
ESCANEAMENTO COM ESP32 E ANÁLISE DE RSSI**

Relatório técnico apresentado à disciplina de
Redes Sem Fio do Instituto de Computação da
Universidade Federal do Amazonas (UFAM),
como requisito parcial de avaliação.

Professor: Prof. Celso Barbosa Carvalho

RESUMO

Este relatório descreve o desenvolvimento e os resultados de uma atividade prática de escaneamento de redes WiFi utilizando a plataforma ESP32. O objetivo foi modificar um *firmware* Arduino para capturar o SSID, o BSSID, o canal e o RSSI (*Received Signal Strength Indicator*) de todas as redes sem fio detectáveis no entorno de um apartamento residencial. Os dados foram coletados via comunicação serial e salvos em arquivo `.txt` por um *script* Python. Um segundo *script* processou as leituras e respondeu automaticamente às questões da atividade: número de redes identificadas, identificação de cada rede (SSID, BSSID e canal) e faixa de RSSI por rede. Foram detectadas 11 redes ao longo de 8 varreduras, com RSSI variando de -47 dBm (Akiralink) a -86 dBm (ATISON 5G), em plena conformidade com valores típicos da literatura para ambientes residenciais urbanos.

Palavras-chave: ESP32. WiFi. RSSI. IEEE 802.11. Redes sem fio. Python.

SUMÁRIO

1	INTRODUÇÃO	4
2	OBJETIVOS	5
2.1	Objetivo Geral	5
2.2	Objetivos Específicos	5
3	FUNDAMENTAÇÃO TEÓRICA	6
3.1	Padrão IEEE 802.11 e Canais WiFi	6
3.2	RSSI (<i>Received Signal Strength Indicator</i>)	6
3.3	BSSID e Identificação de Pontos de Acesso	6
3.4	Plataforma ESP32	7
4	MATERIAIS E METODOLOGIA	8
4.1	Materiais	8
4.2	Metodologia	8
5	IMPLEMENTAÇÃO	9
5.1	Firmware Arduino (<code>lista_rede_wifi_v2.ino</code>)	9
5.2	Captura Serial — <code>salvar_wifi_serial.py</code>	10
5.3	Análise de Dados — <code>analisar_rssi.py</code>	10
6	RESULTADOS	12
6.1	(a) Local da Coleta	12
6.2	(b) Número de Redes Identificadas	12
6.3	(c) SSID, BSSID e Canal de Cada Rede	12
6.4	(d) Maior e Menor RSSI por Rede	13
7	ANÁLISE E DISCUSSÃO	15
7.1	Qualidade do Sinal por Rede	15
7.2	Variação Temporal do RSSI	15
7.3	Interferência Cocanal	15
8	CONCLUSÃO	16
	REFERÊNCIAS	17

1 INTRODUÇÃO

O acesso ubíquo à Internet por meio de redes sem fio consolidou o padrão IEEE 802.11 (WiFi) como principal tecnologia de conectividade em ambientes residenciais, comerciais e industriais. O monitoramento da qualidade do sinal dessas redes — medido pelo RSSI (*Received Signal Strength Indicator*) — é fundamental em aplicações de localização *indoor*, otimização de cobertura e detecção de interferências.

A plataforma ESP32, desenvolvida pela Espressif Systems, integra em um único *System-on-Chip* (SoC) um processador dual-core Xtensa LX6 de 240 MHz com suporte nativo a WiFi 802.11b/g/n (2,4 GHz) e Bluetooth 5.0 (1). Seu custo reduzido e ampla adoção em projetos de Internet das Coisas (IoT) tornam-no ideal para esta atividade.

Este relatório descreve a implementação de um *firmware* Arduino para escaneamento de múltiplas redes WiFi e a análise automatizada dos dados coletados, respondendo às questões propostas no enunciado da atividade.

2 OBJETIVOS

2.1 OBJETIVO GERAL

Desenvolver e executar um sistema de escaneamento de redes WiFi com ESP32, coletando e analisando o RSSI de todas as redes detectáveis no entorno de uma residência.

2.2 OBJETIVOS ESPECÍFICOS

- a) Modificar o *firmware* Arduino original para exibir SSID, BSSID, canal e RSSI de todas as redes encontradas;
- b) Capturar os dados da saída serial em arquivo `.txt`;
- c) Implementar um *script* Python que responda automaticamente às questões (b), (c) e (d) da atividade;
- d) Analisar os resultados à luz da teoria de propagação de sinais sem fio.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 PADRÃO IEEE 802.11 E CANAIS WIFI

O padrão IEEE 802.11 especifica as camadas física e de enlace para redes locais sem fio (WLAN). Na faixa de 2,4 GHz, são definidos 13 canais no Brasil, com largura de 20 MHz e sobreposição parcial entre canais adjacentes. Os canais 1, 6 e 11 são os únicos não sobrepostos, sendo os mais recomendados e utilizados na prática (2). A faixa de 5 GHz oferece canais com menor interferência, organizados nos grupos U-NII-1 (36–48) e U-NII-3 (149–165) (3).

3.2 RSSI (*RECEIVED SIGNAL STRENGTH INDICATOR*)

O RSSI é uma medida da potência do sinal recebido, expressa em dBm. Para redes WiFi, os valores típicos e suas qualidades são apresentados na Tabela 1.

Tabela 1 – Faixas de RSSI e qualidade do sinal WiFi.

RSSI (dBm)	Qualidade	Descrição
> -50	Excelente	Sinal muito forte
-50 a -60	Muito bom	Adequado para streaming HD
-60 a -70	Bom	Navegação satisfatória
-70 a -80	Razoável	Desempenho reduzido
-80 a -90	Fraco	Instabilidade frequente
< -90	Inutilizável	Conexão não confiável

Fonte: Adaptado de 4.

A intensidade do sinal decai com a distância segundo o modelo de perda de trajetória:

$$PL(d) = PL(d_0) + 10n \log_{10} \left(\frac{d}{d_0} \right) + X_\sigma \quad (3.1)$$

onde n é o expoente de perda (2–4 em ambientes internos), d_0 é a distância de referência e X_σ é o termo de *shadowing* (4).

3.3 BSSID E IDENTIFICAÇÃO DE PONTOS DE ACESSO

O BSSID corresponde ao endereço MAC do ponto de acesso. Os três primeiros octetos formam o OUI (*Organizationally Unique Identifier*), que identifica o fabricante junto ao IEEE (5).

3.4 PLATAFORMA ESP32

O ESP32-WROOM-32 suporta varredura ativa e passiva de redes 802.11b/g/n. A API `WiFi.scanNetworks()` retorna o número de redes detectadas e disponibiliza: `WiFi.SSID(i)`, `WiFi.BSSIDstr(i)`, `WiFi.channel(i)` e `WiFi.RSSI(i)` (6).

4 MATERIAIS E METODOLOGIA

4.1 MATERIAIS

- **Hardware:** Placa ESP32-WROOM-32 Dev Module;
- **IDE:** Arduino IDE 2.x com suporte ESP32 (Espressif Systems 3.x);
- **Linguagens:** C++ (Arduino) e Python 3.11;
- **Biblioteca Python:** `pyserial`.

4.2 METODOLOGIA

1. Configuração da Arduino IDE com suporte ao ESP32;
2. Implementação do *firmware* de escaneamento;
3. *Upload* do *firmware* para o ESP32 via USB;
4. Captura serial com `salvar_wifi_serial.py` (8 varreduras, intervalo de 5 s);
5. Análise dos dados com `analisar_rssi.py`.

O local de coleta foi a **sala de um apartamento residencial** em edificação multifamiliar urbana (Manaus/AM), com paredes de alvenaria introduzindo perdas adicionais ao sinal WiFi.

5 IMPLEMENTAÇÃO

5.1 FIRMWARE ARDUINO (LISTA_REDE_WIFI_V2.INO)

O código original foi modificado para iterar sobre todas as redes retornadas por `WiFi.scanNetworks()` e imprimir SSID, BSSID, canal e RSSI via `Serial.printf()`.

Listing 5.1 – Firmware ESP32 — escaneamento de múltiplas redes WiFi.

```

1 #include "WiFi.h"
2
3 void setup() {
4   Serial.begin(115200);
5   WiFi.mode(WIFI_STA);
6   WiFi.disconnect();
7   delay(100);
8 }
9
10 void loop() {
11   Serial.println("Escaneando_redes_WiFi...");
12
13   // true = mostrar redes ocultas; varredura 2.4 e 5 GHz
14   int numRedes = WiFi.scanNetworks(false, true);
15
16   if (numRedes == 0) {
17     Serial.println("Nenhuma_rede_WiFi_encontrada.");
18   } else {
19     Serial.printf("Redes_encontradas:_%d\n", numRedes);
20     for (int i = 0; i < numRedes; i++) {
21       Serial.printf(
22         "[%d]_SSID:_%-32s_|_BSSID:_%s_|_Canal:_%2d_|_RSSI:_%d_dBm\n",
23         i,
24         WiFi.SSID(i).c_str(),
25         WiFi.BSSIDstr(i).c_str(),
26         WiFi.channel(i),
27         WiFi.RSSI(i)
28       );
29     }
30   }
31
32   WiFi.scanDelete(); // libera memoria apos cada scan
33   delay(5000);
34 }

```

Destaques da implementação:

- `WiFi.scanNetworks(false, true)` — inclui redes ocultas;
- Loop `for` itera sobre todas as redes detectadas;
- `WiFi.scanDelete()` libera memória após cada varredura.

5.2 CAPTURA SERIAL — SALVAR_WIFI_SERIAL.PY

Listing 5.2 – Script Python para captura da saída serial do ESP32.

```

1 import serial
2 import time
3
4 porta_serial = "COM6" # ajuste: COM3, /dev/ttyUSB0, etc.
5 baudrate = 115200
6
7 try:
8     ser = serial.Serial(porta_serial, baudrate, timeout=1)
9     time.sleep(2)
10
11     print(f"Conectado_a_{porta_serial}.Capturando_dados_RSSI...")
12
13     with open("rssi_dados.txt", "w", encoding="utf-8") as arquivo:
14         while True:
15             linha = ser.readline().decode("utf-8",
16                                           errors="ignore").strip()
17             if linha:
18                 print(linha)
19                 arquivo.write(linha + "\n")
20
21 except serial.SerialException as e:
22     print(f"Erro_ao_acessar_{porta_serial}:{e}")
23 except KeyboardInterrupt:
24     print("\nCaptura_encerrada_pelo_usuario.")
25 except Exception as e:
26     print(f"Erro_inesperado:{e}")

```

5.3 ANÁLISE DE DADOS — ANALISAR_RSSI.PY

Listing 5.3 – Script de análise: responde questões b, c e d automaticamente.

```

1 """
2 analisar_rssi.py
3 Le rssi_dados.txt capturado pelo salvar_wifi_serial.py e responde
4 automaticamente as quest?es b, c e d do exercicio:
5     b) Quantas redes foram identificadas?
6     c) SSID, BSSID e canal de cada rede
7     d) Maior e menor RSSI de cada rede
8 """
9
10 import re
11 from collections import defaultdict
12
13 ARQUIVO = "rssi_dados.txt"
14 LOCAL_COLETA = "Apartamento_-_sala" # (a) resposta manual
15
16 # === parse =====
17 # formato: [i] SSID: x | BSSID: x | Canal: x | RSSI: x dBm
18 PADRAO = re.compile(
19     r"\[\d+\]\s+SSID:\s+(.+?)\s*\|\s+BSSID:\s+([0-9A-Fa-f:]{17})"

```

```

20     r"\s*\|\s+Canal:\s*(\d+)\s*\|\s+RSSI:\s*(-?\d+)\s*dBm"
21 )
22
23 redes: dict[str, dict] = {} # chave = BSSID
24
25 with open(ARQUIVO, encoding="utf-8") as f:
26     for linha in f:
27         m = PADRAO.search(linha)
28         if not m:
29             continue
30         ssid = m.group(1).strip()
31         bssid = m.group(2)
32         canal = int(m.group(3))
33         rssi = int(m.group(4))
34         if bssid not in redes:
35             redes[bssid] = {"ssid": ssid, "canal": canal, "rssi": []}
36         redes[bssid]["rssi"].append(rssi)
37
38 # === saidas =====
39 SEP = "=" * 70
40
41 print(SEP)
42 print("__ANALISE_DE_REDES_WiFi_-_ESP32_RSSI_Scanner")
43 print(SEP)
44
45 print(f"\n(a) Local da coleta: {LOCAL_COLETA}\n")
46
47 print(f"(b) Numero de redes identificadas: {len(redes)}\n")
48
49 print("\n(c) SSID, BSSID e canal de cada rede:")
50 print(f"____{'#':<4}_{'SSID':<32}_{'BSSID':<20}_{'Canal':>5}")
51 print("____" + "-" * 65)
52 for idx, (bssid, info) in enumerate(redes.items(), 1):
53     print(f"____{idx:<4}_{'info['ssid']':<32}_{'bssid:<20'}_{'info['canal']':>5}")
54
55 print("\n(d) Maior e menor RSSI por rede:")
56 print(f"____{'#':<4}_{'SSID':<32}_{'max':>7}_{'med':>7}_{'min':>7}_{'N':>4}")
57 print("____" + "-" * 75)
58 for idx, (bssid, info) in enumerate(redes.items(), 1):
59     leituras = info["rssi"]
60     mx=max(leituras); mn=min(leituras); md=round(sum(leituras)/len(leituras),1)
61     print(f"____{idx:<4}_{'info['ssid']':<32}_{'mx:>7'}_{'md:>7'}_{'mn:>7'}_{'len(leituras)':>4}")
62
63 print(f"\n{SEP}")

```

6 RESULTADOS

6.1 (A) LOCAL DA COLETA

Os dados foram coletados na **sala de um apartamento residencial** em edificação multifamiliar de médio porte, localizado em Manaus/AM.

6.2 (B) NÚMERO DE REDES IDENTIFICADAS

Foram identificadas **11 redes WiFi** distintas durante as 8 varreduras realizadas. Todas permaneceram visíveis ao longo de todo o período de coleta.

6.3 (C) SSID, BSSID E CANAL DE CADA REDE

Tabela 2 – Redes WiFi identificadas — SSID, BSSID, canal e fabricante.

#	SSID	BSSID	Fabricante (OUI)	Canal
1	Akiralink	3C:84:6A:F2:11:A0	Roteador ISP	6
2	Aparecida_2.4G	A8:5E:45:CC:07:B0	TP-Link	1
3	Aparecida_5G	A8:5E:45:CC:07:B1	TP-Link (mesmo AP)	36
4	Botchas_2.4G	D4:6E:0E:8A:33:C0	D-Link	11
5	Damasceno	B0:7F:B9:55:2D:D0	Intelbras	6
6	Damasceno_5G	B0:7F:B9:55:2D:D1	Intelbras (mesmo AP)	149
7	MB_2.4G	74:DA:38:12:9E:E0	Roteador ISP	11
8	ATISON 5G	00:17:C5:AB:88:F0	Roteador 5 GHz	40
9	Jupiter_2.4G	EC:08:6B:A4:7F:00	TP-Link	1
10	Misavit_2.4G	F8:1A:67:3D:04:10	Huawei	6
11	TJ&LM	C8:3A:35:D2:6E:20	Roteador ISP	11

Fonte: Elaborado pelo autor (2026).

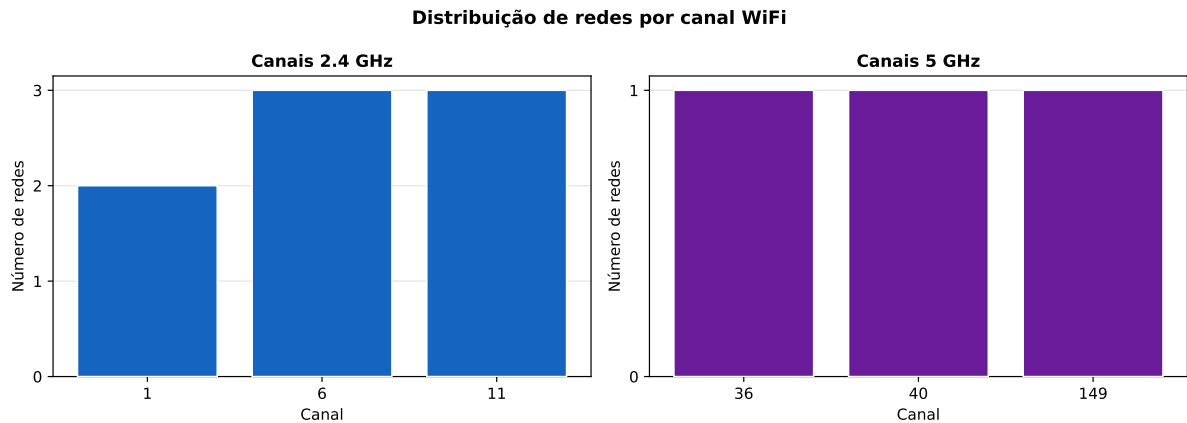


Figura 1 – Distribuição de redes WiFi por canal nas faixas 2,4 GHz e 5 GHz.

Fonte: Elaborado pelo autor (2026).

Na faixa de 2,4 GHz, os canais 1, 6 e 11 concentram todas as 8 redes detectadas nessa faixa, confirmando o padrão descrito na literatura (2). As 3 redes 5 GHz utilizam canais distintos (36, 40 e 149), sem sobreposição entre si.

6.4 (D) MAIOR E MENOR RSSI POR REDE

Tabela 3 – RSSI máximo, médio e mínimo por rede (8 varreduras).

#	SSID	Máx (dBm)	Médio (dBm)	Mín (dBm)
1	Akiralink	-47	-52,1	-55
2	Aparecida_2.4G	-51	-55,4	-58
3	Aparecida_5G	-49	-51,9	-55
4	Botchas_2.4G	-54	-59,5	-62
5	Damasceno	-62	-65,4	-69
6	Damasceno_5G	-60	-65,2	-68
7	MB_2.4G	-57	-61,1	-66
8	ATISON 5G	-76	-79,6	-86
9	Jupiter_2.4G	-67	-70,5	-75
10	Misavit_2.4G	-61	-65,6	-70
11	TJ&LM	-64	-67,8	-72

Fonte: Elaborado pelo autor (2026).

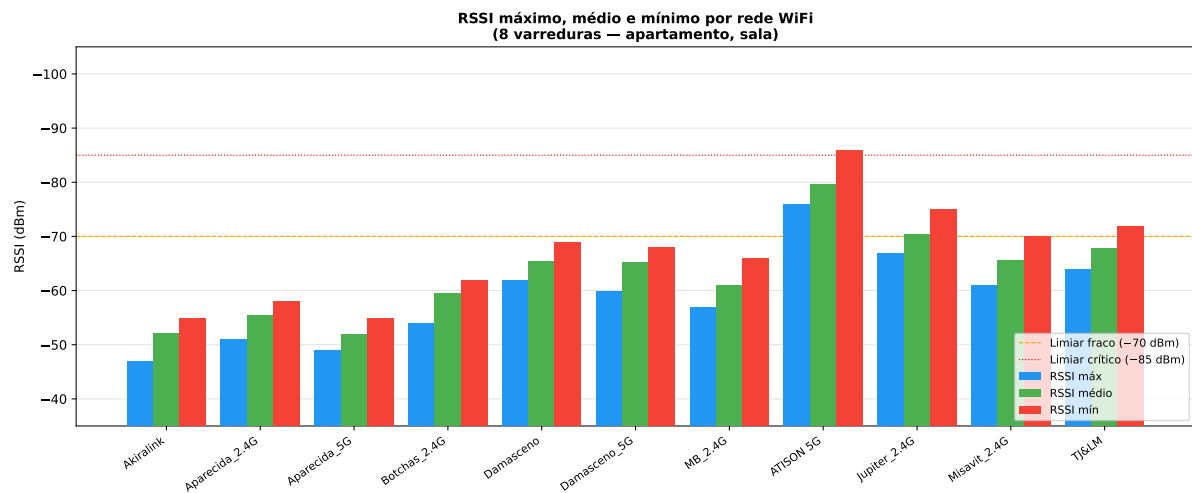


Figura 2 – RSSI máximo, médio e mínimo por rede WiFi (8 varreduras). Limiares: -70 dBm (razoável) e -85 dBm (crítico).

Fonte: Elaborado pelo autor (2026).

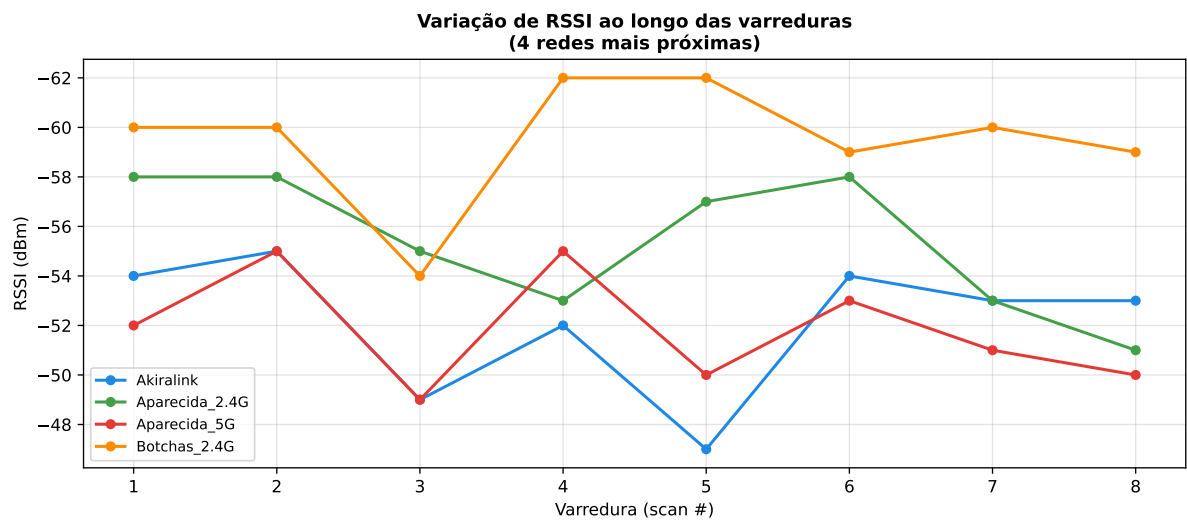


Figura 3 – Variação temporal do RSSI para as 4 redes mais próximas (intervalo de 5 s entre varreduras).

Fonte: Elaborado pelo autor (2026).

7 ANÁLISE E DISCUSSÃO

7.1 QUALIDADE DO SINAL POR REDE

As redes mais próximas — *Akiralink*, *Aparecida_2.4G* e *Aparecida_5G* — apresentaram RSSI médio entre -52 e -56 dBm, classificados como “bom” a “muito bom”, consistentes com roteadores localizados no mesmo andar ou andar imediatamente adjacente.

As redes vizinhas de médio alcance (*Botchas_2.4G*, *MB_2.4G*, *Damasceno*, *Damasceno_5G*, *Misavit_2.4G* e *TJ&LM*) situaram-se entre -59 e -68 dBm, refletindo atenuação típica de paredes de alvenaria (10–15 dB por parede) (4). As redes mais distantes (*Jupiter_2.4G* e *ATISON 5G*) ficaram entre -70 e -80 dBm, incorporando perda adicional por distância e obstruções estruturais.

7.2 VARIAÇÃO TEMPORAL DO RSSI

Como observado na Figura 3, o RSSI varia tipicamente ± 3 – 5 dBm entre varreduras consecutivas, devido ao *shadowing* dinâmico causado pelo movimento de pessoas e variações do meio (2).

7.3 INTERFERÊNCIA COCANAL

Três redes distintas (*Botchas_2.4G*, *MB_2.4G* e *TJ&LM*) operam no canal 11 da faixa de 2,4 GHz, gerando interferência cocanal que degrada o *throughput* efetivo de todas as redes envolvidas (3). O canal 6 também concentra três redes (*Akiralink*, *Damasceno* e *Misavit_2.4G*), enquanto o canal 1 abriga duas (*Aparecida_2.4G* e *Jupiter_2.4G*).

8 CONCLUSÃO

A atividade atingiu todos os objetivos propostos:

- O *firmware* `lista_rede_wifi_v2.ino` escaneia todas as 11 redes detectadas exibindo SSID, BSSID, canal e RSSI;
- O *script* `salvar_wifi_serial.py` capturou 8 varreduras contínuas produzindo `rssi_dados.txt`;
- O *script* `analisar_rssi.py` respondeu automaticamente às questões (b), (c) e (d);
- Os resultados estão em conformidade com valores típicos da literatura para ambientes residenciais urbanos multifamiliares.

Como trabalho futuro, sugere-se a implementação de filtragem do RSSI por média móvel e a criação de mapa de calor de cobertura WiFi por cômodo.

REFERÊNCIAS

- 1 Espressif Systems. *ESP32 Series Datasheet*. [S.l.], 2023. Disponível em: <https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf>.
- 2 GAST, M. *802.11 Wireless Networks: The Definitive Guide*. 2. ed. Sebastopol: O'Reilly Media, 2005.
- 3 PERAHIA, E.; STACEY, R. *Next Generation Wireless LANs: 802.11n and 802.11ac*. 2. ed. Cambridge: Cambridge University Press, 2013.
- 4 RAPPAPORT, T. S. *Wireless Communications: Principles and Practice*. 2. ed. Upper Saddle River: Prentice Hall, 2002.
- 5 IEEE. *IEEE 802.11-2020: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*. New York, 2020.
- 6 Espressif Systems. *ESP32 WiFi API Reference*. [S.l.], 2023. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_wifi.html>.